

PARLEY

Design Brief & Functional Specification

AAC Reply Copilot for iPad — Built for James

Product	Parley AAC Companion
Platform	iPad-first PWA (React 19 + TanStack Start)
Backend	Lovable Cloud (Supabase) + IndexedDB local-first
Voice Pipeline	ElevenLabs Scribe (STT) + ElevenLabs TTS
AI Engine	Lovable AI Gateway (Gemini 2.5 Flash / Pro via OpenAI-compatible API)
On-device ML	Meyda MFCC speaker fingerprinting (no cloud)
Version	Tier 3.1 — Semantic Memory Retrieval

1. Purpose & Objective

Parley is an iPad-first Augmentative and Alternative Communication (AAC) copilot designed to help a non-speaking or speech-limited user — James — participate in live conversations. It listens to the room, transcribes speech in real time, and suggests contextually-aware reply options that James can tap to speak aloud via text-to-speech.

Core Design Principles

Speed: Suggestions must appear within 1–2 seconds of a speaker finishing. Every millisecond of latency is felt by James.

Privacy: All voice fingerprinting runs on-device via MFCC (Meyda). No speaker audio leaves the device for identification.

Character consistency: The more James uses Parley, the better it learns his vocabulary, humour, formality, and taboo phrases — creating a style profile that makes suggestions sound increasingly like him.

Context awareness: Parley knows who is in the room, where the conversation is happening, and what event is planned — surfacing memories, follow-ups, and conversation points at the right moment.

Primary Users

James (end user): Taps suggestions, types rough text, or speaks via TTS. Needs large touch targets, high contrast, and minimal cognitive load.

Carer / family member: Sets up people, places, events, and James's profile in Settings. May record voice samples for frequent contacts.

2. Application Architecture

Parley is structured as a landscape-oriented iPad cockpit. The screen is divided into functional zones rather than traditional pages. Navigation between major areas is handled by a bottom tab bar or side buttons.

Technology Stack

Frontend: React 19, TanStack Start v1 (file-based routing), Tailwind CSS v4 with custom oklch design tokens.

Local Database: Dexie (IndexedDB wrapper) with 9 schema versions — stores conversations, transcripts, people, places, events, memories, voiceprints, style profiles, and settings.

Cloud Sync: Lovable Cloud (Supabase) — auth, user accounts, and automatic backup/restore of IndexedDB contents.

Server Functions: TanStack createServerFn — typed RPC for AI calls, TTS, voice design, and Scribe token generation.

STT Pipeline: ElevenLabs Scribe (real-time streaming transcription via React hook).

TTS Pipeline: ElevenLabs Turbo v2.5 — MP3 output, base64-embedded playback.

Speaker ID: On-device MFCC extraction (20 coefficients) via Meyda, cosine-similarity matching, centroid-based clustering with 0.82 threshold.

Major Functional Areas

- **Home / Cockpit (/)** — Live conversation — recording, transcript, suggestions, speaker panel, mood selector
- **Recent Conversations (/recent)** — Browse, search, and filter past conversations with summaries and highlights
- **Reply Helpers (/helpers)** — Draft replies for iMessage, Email, and Facebook in James's voice
- **Settings (/settings)** — Five-tab admin: About James, People, Locations, Events, System

3. Home / Live Conversation Cockpit (/)

This is the primary screen. It operates in two modes: **Idle** (before recording) and **Active** (during recording). The layout is designed for landscape iPad with the screen split into functional zones.

3.1 Idle Mode (Before Recording)

When the app first loads, James sees the main cockpit in its pre-recording state. The top section contains the transcript area (empty or showing 'Listening...'), and the main area shows the suggestion grid.

Suggestion Grid (3 x 3)

A 3x3 grid of large, tappable suggestion cards dominates the central area. Each card displays:

- **Primary text** — the suggested reply (e.g. 'That's a great idea, I'm in!')
- **Category badge** — coloured indicator showing whether this is an Answer, Question, Follow-up, Planned point, Humour, or Clarify suggestion
- **Optional 'why' tooltip** — explaining context (shown on long-press or as smaller text)

The grid is fed by `generateSuggestions` (server function) which takes the current conversation context, recent transcript, and mood as inputs. Suggestions refresh automatically every 1.5 seconds when new transcript arrives.

Mood Selector (Bottom Bar)

A horizontal row of large mood buttons sits along the bottom of the screen. Options: Normal, Calm, Excited, Sad, Upset, Empathetic, Amused. Each mood has a distinct background colour. Tapping a mood changes the tone of future AI-generated suggestions and spoken replies. The selected mood is passed to `generateSuggestions` as a parameter.

Quick Phrases (Bottom-Left)

Five always-visible phrase buttons for the most common needs: 'Yes', 'No', 'Give me a moment', 'Could you repeat that?', 'Sorry, who am I speaking with?'. These bypass the AI suggestion pipeline and speak immediately via TTS when tapped.

3.2 Active Mode (During Recording)

When James presses the large circular Record button, the app enters Active mode. ElevenLabs Scribe begins streaming real-time transcription.

Record / Stop Button

- **Record (red circle with Mic icon)**: Tapping starts the conversation. Requests microphone permission on first use. Generates a single-use Scribe token via `createScribeToken` server function.
- **Stop (square icon)**: Stops recording. Triggers the post-conversation pipeline (Tier 2): re-diarization, voiceprint rebuilding, profile enrichment, and introduction detection. Also generates a conversation summary via `summarizeConversation`.

Transcript Panel (Left Side)

Shows the last ~30 transcript segments in a scrollable panel. Each line shows:

- **Speaker name** — resolved from the Speaker Panel (e.g. 'Mum', 'Matt', 'Me')
- **Text** — the transcribed speech

- **Partial text** — italic grey text while Scribe is still committing

Each line is tappable to reassign who said it (correction flow). This updates the underlying segment's `person_id` in IndexedDB.

Speaker Panel (Right Side)

Divided into two sub-panels:

Live Transcript (top): Same content as the left panel but focused on speaker management.

Speakers (bottom): The speaker roster. Each detected speaker cluster appears as a card with one of three states:

- **Unknown** — grey border, shows suggested name chips from self-intros or 'Ask' replies
- **Suggested** — amber border, 'Sounds like Mum (87%)' with Confirm / Not them / It's someone else actions
- **Confirmed** — emerald border, name displayed with edit (reassign/clear) option

Speakers are identified via on-device MFCC fingerprinting against stored voiceprints. The diarizer clusters incoming audio with a cosine-similarity threshold of 1.82. Two consecutive matches at ≥ 0.80 similarity are required before suggesting a known person.

3.3 Typed-Text Expansion Flow

James can type rough text into a text area at the bottom. The Expand button sends the text to `expandUtterance` (server function) which returns a polished, grammatically-complete version in James's voice. The expanded text is then spoken aloud via TTS. Both the raw and expanded versions are stored in the transcript as 'Me' entries.

3.4 Context Setup (Before Recording)

Before starting, James (or a carer) can configure:

- **People picker** — select from stored people; shows green badge if voiceprint exists, amber if not. Inline voice sample recorder available.
- **Place** — auto-detected via GPS (if enabled) or manually selected. Affects suggestion context.
- **Event** — choose an upcoming event to surface key points and questions during the conversation.

4. Recent Conversations (/recent)

A browseable history of all recorded conversations. Accessed via the History icon in the top navigation.

Filter & Search Bar

- **Search input** — free-text search across summaries, highlights, place names, and person names
- **Person filter dropdown** — filter to conversations involving a specific person
- **Place filter dropdown** — filter to conversations at a specific location
- **Sort dropdown** — Newest first, Oldest first, By location, By person
- **Clear button** — resets all filters instantly

Conversation Cards

Each card displays:

- Date/time of the conversation
- Place name (if recorded)
- Confirmed people names
- Summary paragraph (if generated)
- Bullet list of highlights (if generated)

Cards are ordered by start time (descending by default). The underlying data comes from the conversations IndexedDB table joined with people and places.

5. Reply Helpers (/helpers)

A drafting workbench for asynchronous communication. Unlike the live conversation cockpit, this is for composing replies to messages, emails, and social media posts when James is not in a face-to-face conversation.

Three Platform Tabs

Messages (iMessage) — Incoming text pasted into a box; James types a rough reply; AI drafts a polished version. Copy and paste into Messages app.

Email (Gmail) — Same flow but with email tone. Includes an 'Open Gmail' button that launches Gmail in a popup window side-by-side.

Facebook — Supports four post types: Status update, Comment, Reply to comment, Messenger message. Post type selector changes whether the 'incoming' box is shown.

Draft Helper Component

The DraftHelper component provides:

- **Incoming text area** — what James is replying to (optional for status updates)
- **Draft text area** — James's rough shorthand
- **Context input** — freeform context like 'from his son Jack'
- **Generate button** — fires draftReply server function with platform, incoming text, raw draft, James's profile, and selected model
- **Recommended output** — primary polished reply shown in a card
- **Alternatives** — 2–3 variations with different tones
- **Copy button** — one-tap copy to clipboard
- **Interest suggestions** — AI proposes topics James might want to mention, surfaced after drafting

6. Settings (/settings)

Five-tab administrative interface for configuring James's profile, people, places, events, and system preferences. Accessed via the Settings icon.

6.1 About James

James's personal profile — the foundation of all suggestion generation. Fields include:

- **Display name** — used in prompts and UI labels
- **Age** — freeform text
- **Background** — family, career, where he grew up
- **Personality** — dry wit, warm, reserved, etc.
- **Humour style** — sarcastic, gentle, observational
- **Communication style** — short sentences, prefers questions, formal/informal
- **Topics loved** — sailing, cricket, music, etc.
- **Topics avoided** — sensitive subjects to steer AI away from
- **Signature phrases** — catchphrases James uses (one per line)
- **Current life context** — recent events, what's on his mind
- **Freeform notes** — anything else

Profile Documents: Upload reference documents (PDFs, text files) about James. Content is extracted and folded into the suggestion prompt as context.

6.2 People

CRUD management for everyone James knows. Each person record contains:

- **Name** and **relationship** to James
- **Interests** — comma-separated tags
- **Notes** — freeform observations
- **Style notes** — how this person speaks
- **Topics loved / avoided** — auto-enriched by AI from conversations
- **Relationship dynamics** — freeform observations about the James↔person dynamic
- **Dynamic tags** — constrained tags like 'teases', 'interrupts'
- **Voiceprint status** — learned sample count or 'no voiceprint yet'

Voice Sample Recorder: Each person detail panel includes a 'Record voice sample' section. Tap Record → speak for 3–8 seconds → MFCC extracted → saved to voiceprints table. Re-recording adds to the centroid via running mean. Replace starts fresh. Delete removes all contributions.

Auto-detected people: The 'Ask' flow and self-introduction detection can propose new people. These appear with status='auto' and await confirmation in Settings.

6.3 Locations

Places James visits. Each place has:

- **Name**
- **GPS coordinates** (lat, lng) with radius
- **Notes**

MapPicker component: An interactive map where carers drop a pin, set a radius circle, and name the

place. GPS auto-detection (if enabled) suggests the nearest known place when starting a conversation.

6.4 Events

Upcoming social events James is attending. Each event has:

- **Name, date/time, location**
- **Attending people** — selected from People list
- **Key info** — freeform context
- **Prep prompt** — user-provided steering for AI preparation
- **Key points** — auto-generated by AI; selectable checkboxes
- **Key questions** — auto-generated; selectable checkboxes
- **Notes**
- **Documents** — uploaded event-related files

Event Prep generation: The 'Generate prep' button calls generateEventPrep (server function) which analyses the event context and produces talking points and questions. These are surfaced during live conversations.

6.5 System

Technical and preference settings:

Voice: Select TTS voice from ElevenLabs male voices. Preview button plays 'Hi, this is how I'll sound...'. Voice Designer panel allows creating custom voices via text description.

AI Models:

- **Fast model** — powers live suggestions (Gemini 2.5 Flash Lite default). James feels every extra second.
- **Smart model** — powers summaries, memory extraction, event prep, drafts (Gemini 2.5 Pro default). Quality over speed.

Display Size: iPad model selector (Mini, 10.9', Air/Pro 11', Pro 12.9', Pro 13'). Scales the UI to fit without scrolling.

Location: GPS toggle for automatic place detection.

Storage & Privacy: Explanation of IndexedDB + cloud backup. 'Clear all local data' button with destructive confirmation dialog.

7. Engine Room — Key Functions & Libraries

This section details the core functions that power Parley, organised by library file.

7.1 aac.functions.ts — Server Functions

createScribeToken() — Generates a single-use ElevenLabs Scribe token for real-time transcription. Called when Record is pressed.

synthesizeSpeech({ text, voiceId }) — Converts text to speech via ElevenLabs Turbo v2.5. Returns base64 MP3. Called when any suggestion or phrase is tapped.

generateSuggestions({ context, transcript, mood, ... }) — The heart of the app. Takes full conversation context (James profile, people, place, event, style evidence, memories) and recent transcript, returns 6–9 suggestion objects with text, category, and optional 'why'. Uses the Fast model via Lovable AI Gateway.

summarizeConversation({ conversationId, segments, ... }) — Post-stop summary generation using the Smart model. Produces a paragraph summary and bullet highlights.

expandUtterance({ rawText, context, voiceNotes, model }) — Takes James's rough typed text and expands it into a polished, grammatically complete sentence in his voice.

identifySpeakerFromContext({ transcript, people, ... }) — AI fallback for speaker identification when MFCC matching is ambiguous. Analyses word choice, topics, and references.

draftReply({ platform, incoming, rawText, context, jamesProfile, model }) — Generates polished replies for Messages/Email/Facebook.

extractInterests({ draft, incoming, ... }) — Analyses a drafted reply and suggests topics James might want to mention, based on his profile.

designVoicePreviews({ description }) + saveDesignedVoice({ ... }) — ElevenLabs Voice Design API integration. Generate and save custom voices.

generateEventPrep({ event, people, model }) — Produces key points and questions for an upcoming event.

aiRediarizeTieBreaker({ ... }) — LLM-based speaker disambiguation during post-conversation re-diarization.

enrichPersonProfile({ ... }) — Post-conversation AI analysis that proposes updates to a person's profile (topics loved, avoided, dynamics).

detectIntroductions({ ... }) — Scans transcript for self-introductions and proposes new Person records.

7.2 voiceprint.ts — On-Device Speaker ID

VoiceCapture class — Captures microphone PCM via ScriptProcessorNode. Keeps a rolling 5-minute buffer. Supports shift monitoring (detects speaker changes by comparing MFCC cosine similarity every 200ms).

computeMfccMean(pcm, sampleRate) — Extracts 20 MFCC coefficients via Meyda, averaged across all non-silent frames.

cosineSim(a, b) — Normalised cosine similarity between two MFCC vectors.

discriminativeSim(a, b) — Weighted cosine similarity that emphasises the most discriminative coefficients.

Diarizer class — Online clustering engine. `assignNew()` creates a cluster; `assign(label, mfcc)` adds to existing; `forceAssign()` overrides. Threshold 0.82 for cluster membership. Tracks running centroids.

recordVoiceprint(personId, mfcc) — Updates a person's voiceprint centroid via running mean.

bestMatch(mfcc, candidates) — Finds the best-matching stored voiceprint.

rebuildVoiceprintFromContributions(personId) — Re-clusters all contributions for a person (k-means with up to 2 sub-centroids) and writes back a sharper centroid with confidence score.

7.3 context.ts — Context Builder

buildConversationContext({ personIds, place, event, queryEmbedding }) — Assembles the full context object fed to AI prompts. Includes:

- James profile (structured + freeform + documents)
- Each person's details, recent memories, follow-ups, and uploaded documents
- Place with recent memories and follow-ups
- Event with key points, questions, and documents
- Style profile JSON
- Style evidence (Tier 1 feedback loop)
- Semantic memory retrieval (when query embedding supplied)

Results are cached for 5 minutes to avoid redundant IndexedDB queries.

suggestPeopleAtPlace(placeId) — Returns people most frequently seen at a given place, for pre-populating the people picker.

7.4 post-conversation.ts — Tier 2 Pipeline

Runs after Stop is pressed, in this order:

1. rediarizeAfterStop — Re-clusters all non-James transcript segments using stored voiceprints as seeds. k-means with cosine distance. Reassigns segments to the most likely speaker. Returns cluster centroids and confidence scores.

2. rebuildVoiceprintsAfterStop — Gathers all MFCC contributions per confirmed person, re-runs k-means (up to 2 sub-centroids), writes back a sharper centroid with confidence score.

3. enrichProfilesAfterStop — AI analysis of the full transcript. Proposes profile updates for each person: topics they loved/avoided, relationship dynamics, dynamic tags. Creates ProfileProposal records awaiting user review in Settings.

4. detectIntroductionsAfterStop — Scans for 'I'm [Name]' / 'This is [Name]' patterns in segments that weren't confirmed to a known person. Proposes new Person records with auto-populated voiceprints from the cluster centroids produced in step 1.

7.5 style-distill.ts & style-evidence.ts — Tier 1 Feedback Loop

Style Evidence tracks every suggestion shown to James: what he tapped, what he edited it to, what he ignored, and what was displaced by a refresh. Aggregated per person.

getCrossSessionDeadPhrases() — Returns phrases that appear frequently but are never selected, to suppress them from future suggestions.

runStyleDistillation() — Periodically (or on-demand) analyses all suggestion logs across sessions to produce a StyleProfile:

- Preferred openers and sign-offs
- Formality level (casual / neutral / formal)
- Humour markers
- Taboo phrases (things James never says)
- Average sentence length
- Reading grade estimate
- Category preference scores

This profile is folded into the suggestion prompt, making AI replies sound progressively more like James.

7.6 db.ts — IndexedDB Schema

Dexie database with the following stores:

- **conversations** — id, started_at, ended_at, person_ids, place_id, summary, highlights, speaker_map
- **transcript_segments** — id, conversation_id, speaker_label, person_id, text, ts, confidence, mfcc, embedding
- **people** — id, name, relationship, interests, notes, style_notes, topics_loved, topics_avoided, status, voiceprint_confidence
- **places** — id, name, lat, lng, radius_m, notes
- **events** — id, name, when, location, person_ids, key_info, key_points[], key_questions[], notes
- **memories** — id, person_id, place_id, conversation_id, text, kind, status, embedding
- **follow_ups** — id, for_person_id, for_place_id, text, used
- **voiceprints** — id==person_id, centroid[], sample_count, confidence, sub_centroids
- **voiceprint_contributions** — id, person_id, conversation_id, source, mfcc[], preview_text
- **suggestions_log** — every suggestion shown: text, category, selected, edited_to, ignored, displaced_at, person_id
- **style_profile** — singleton with distilled JSON
- **style_evidence_cache** — per-person aggregated stats
- **james_profile** — singleton with all profile fields
- **settings** — singleton: voice_id, models, gps, display, etc.
- **manual_replies** — typed-text expansions
- **segment_mfccs** — per-segment MFCC vectors for Tier 2 re-diarization
- **person_documents / james_documents / event_documents** — uploaded file content

8. Data Flow & Connections

This section describes how the major functional areas connect together during a typical usage session.

8.1 Live Conversation Loop

1. **Setup** — James or carer opens the app. If GPS is on, place is auto-detected. People and event may be pre-selected.
2. **Record** — Tap Record. Scribe token generated. Microphone activates. VoiceCapture starts buffering PCM.
3. **Transcription** — Scribe streams partial → committed transcript. Each committed segment triggers `processUtterance()`.
4. **Speaker ID** — MFCC extracted from buffered PCM around the segment timestamp. Diarizer assigns to cluster. Voiceprint matching suggests known people.
5. **Context Build** — `buildConversationContext()` assembles James profile + people + place + event + memories + style evidence (cached).
6. **Suggestion Generation** — `generateSuggestions()` called every 1.5s or on transcript commit. Fast model returns 6–9 suggestions.
7. **Display** — 3×3 grid rendered. James taps a suggestion → `synthesizeSpeech()` → audio plays.
8. **Typed Reply** — James types rough text → `expandUtterance()` → TTS plays. Raw + expanded stored.
9. **Stop** — Tap Stop. `summarizeConversation()` generates summary. Tier 2 pipeline runs: re-diarize, rebuild voiceprints, enrich profiles, detect introductions.
10. **Memory Storage** — AI-extracted memories, follow-ups, and highlights saved to IndexedDB. Cloud sync pushes to Lovable Cloud.

8.2 Settings → Live Conversation

- **People + voiceprints** created in Settings feed the speaker ID pipeline during live conversations.
- **Places** configured in Settings enable auto-detection and location-aware suggestions.
- **Events** with key points/questions are surfaced in suggestion prompts during live conversations.
- **James profile** edits immediately affect suggestion generation on next refresh (cache invalidated).
- **Voice** selection changes TTS output immediately.
- **Model** selection changes which AI model powers suggestions, summaries, and drafts.

8.3 Recent → Live Conversation

- Past conversations provide **summaries and highlights** for the Recent page.
- Past **memories** (extracted from conversations) are retrieved by `context.ts` and shown in suggestion prompts.
- Past **suggestion logs** power the Tier 1 style feedback loop, improving future suggestions.
- Past **speaker maps** and **voiceprints** improve speaker recognition accuracy over time.

8.4 Helpers → Profile Enrichment

- Drafts created in Helpers are not stored as conversations, but the **extractInterests** function analyses them and may propose updates to James's `topics_loved` or `signature_phrases`.
- The same **draftReply** function and **James profile** context used in Helpers is shared with the live `expandUtterance` flow.

9. Design System & Tokens

Parley uses a custom 'Slate & Sun' colour palette defined in `src/styles.css` using `oklch` colour space.

Core Palette

- **--paper** — #faf8f5 — Primary background (warm off-white)
- **--paper-warm** — #f0ebe3 — Card/surface backgrounds
- **--ink** — #1e293b — Primary text (slate navy)
- **--ink-soft** — #475569 — Secondary text, captions
- **--hairline** — #cbd5e1 — Borders, dividers
- **--amber** — #f59e0b — Warm accent — questions, highlights
- **--teal** — #14b8a6 — Primary action — answers, confirmations
- **--teal-deep** — #0f766e — Secondary teal — clarifications
- **--indigo** — #6366f1 — Planned points, structural
- **--sage** — #84cc16 — Follow-ups, growth
- **--coral** — #f97316 — Humour, warmth

Category Accents

Each suggestion category has a distinct colour used as a 15% background tint with a 40% border:

- **Answer** — teal
- **Question** — amber
- **Follow-up** — sage
- **Planned point** — indigo
- **Humour** — coral
- **Clarify** — teal-deep
- **Give-me-a-moment** — muted grey

Typography

Default system font stack. Large text sizes throughout for accessibility:

- Suggestion cards: 16–18px
- Buttons: 14–16px
- Transcript: 14px
- Speaker panel labels: 13px
- Quick phrases: 14px bold

Touch Targets

All interactive elements are designed for motor accessibility:

- Suggestion cards: minimum 100×100px touch targets
- Mood buttons: 60×44px minimum
- Record button: 80×80px

- Quick phrases: 48px height minimum

10. Security & Privacy Model

Authentication: Lovable Cloud auth (email/password or social). AuthGate wraps all routes. Session persisted in localStorage.

Data residency: Primary data lives in IndexedDB on the iPad. Cloud sync pushes encrypted backups to Lovable Cloud. Sign in on another device restores everything.

Voice privacy: Speaker identification (MFCC extraction, clustering, matching) runs entirely in the browser. No audio or MFCC vectors leave the device.

AI privacy: Transcript text is sent to AI APIs (Lovable AI Gateway or OpenAI) for suggestion generation. No audio is sent to AI.

TTS privacy: Text-to-speech uses ElevenLabs API. Only the text being spoken is transmitted.

STT privacy: Real-time transcription uses ElevenLabs Scribe. Audio stream is encrypted and sent to ElevenLabs servers.

RLS: All Supabase tables have Row Level Security enabled. Users can only access their own data.

11. Glossary

- **AAC** — Augmentative and Alternative Communication — tools that help people with speech difficulties communicate
- **MFCC** — Mel-Frequency Cepstral Coefficients — audio features extracted from voice for speaker identification
- **STT** — Speech-to-Text — converting spoken audio to written text (ElevenLabs Scribe)
- **TTS** — Text-to-Speech — converting written text to spoken audio (ElevenLabs Turbo v2.5)
- **Diarization** — The process of identifying 'who spoke when' in an audio stream
- **Voiceprint** — A numerical fingerprint (MFCC centroid) representing a person's voice
- **Centroid** — The mean vector of a cluster — represents the 'average' voice of a speaker
- **Tier 1** — Feedback loop — learning from James's suggestion selections to improve future suggestions
- **Tier 2** — Post-conversation pipeline — re-diarization, voiceprint rebuilding, profile enrichment
- **Tier 3.1** — Semantic memory retrieval — using text embeddings to find relevant past memories
- **Scribe** — ElevenLabs real-time streaming transcription service
- **Lovable AI Gateway** — Managed AI API proxy supporting Gemini, GPT, and other models
- **IndexedDB** — Browser-based NoSQL database for client-side persistence
- **Dexie** — A wrapper library that makes IndexedDB easier to use

12. Appendix: Complete Exported Function List

A reference of all major exported functions across the codebase.

From `aac.functions.ts`

- `createScribeToken()`
- `synthesizeSpeech()`
- `listVoices()`
- `designVoicePreviews()`
- `saveDesignedVoice()`
- `generateSuggestions()`
- `summarizeConversation()`
- `expandUtterance()`
- `identifySpeakerFromContext()`
- `draftReply()`
- `extractInterests()`
- `generateEventPrep()`
- `aiRediarizeTieBreaker()`
- `enrichPersonProfile()`
- `detectIntroductions()`

From `voiceprint.ts`

- `VoiceCapture` class
- `computeMfccMean()`
- `cosineSim()`
- `discriminativeSim()`
- `Diarizer` class
- `recordVoiceprint()`
- `bestMatch()`
- `rebuildVoiceprintFromContributions()`
- `addContributionWithCap()`
- `deleteVoiceprint()`

From `context.ts`

- `buildConversationContext()`

- suggestPeopleAtPlace()
- invalidateContextCache()

From post-conversation.ts

- rediarizeAfterStop()
- rebuildVoiceprintsAfterStop()
- enrichProfilesAfterStop()
- detectIntroductionsAfterStop()

From style-distill.ts & style-evidence.ts

- runStyleDistillation()
- getStyleEvidence()
- getCrossSessionDeadPhrases()

From db.ts

- getSettings()
- updateSettings()
- getJamesProfile()
- updateJamesProfile()
- newId()
- seedJamesIfNeeded()

From retrieval.ts

- retrieveTopK()
- formatMemoryForPrompt()
- generateMemoryEmbedding()

From auto-person.ts

- extractIntroducedNames()

From speaker-id.ts

- labelTranscriptForPrompt()

From rediarize.ts

- kmeansRediarize()